

Securing Ubuntu Using AppArmor



Andrew Mallett

Linux Author and Trainer

@theurbanpenguin www.theurbanpenguin.com



Module Overview



Understanding AppArmor and application security

Installing utilities

Creating profiles

Protecting your own applications



AppArmor



AppArmor is a MAC, Mandatory Access Control, application security system promoted by SUSE and Ubuntu. Many tools are authored by Canonical (Ubuntu) and Novell (ex SUSE Owners)



Works with the Linux Kernel LSM (Linux Security Module). Logs messages from Kernel



Protects even where processes are running as root



```
$ sudo systemctl status apparmor
$ aa-status
$ sudo aa-status
$ sudo aa-status --enforced
$ ls /etc/apparmor.d
$ dpkg -S /etc/apparmor.d/lsb_release
$ apt show apparmor
```

Investigating AppArmor

AppArmor is installed by default on Ubuntu 20.04, although extra utilities may be useful. Apparmor has profiles which can be enforced or complaning.

```
$ sudo apt install -y apparmor-easyprof apparmor-utils apparmor-notify
```

Installing Extra Utilities

We can add additional utilities to help support AppArmor management. Be ware that apparmor-notify will add GUI tools as well but is great to extract logged information.

Demo



Up and running with AppArmor

- install utils
- use aa-status



```
$ sudo nano /usr/bin/file_test.py
#!/usr/bin/python3
FILE = '/tmp/mytextfile'
try:
    open(FILE, 'a').close()
    print(f'Created file: {FILE}')
except:
    print(f'Failed to create file {FILE}')
    exit(1)
$ sudo chmod 755 /usr/bin/file_test.py
$ file_test.py
$ rm /tmp/mytextfile
```

Simple Python Script

We can create a very simple script to test AppArmor. Don't forget to make the script executable. Without a profile it can run unconfined; however, we can also create a profile to ensure only the correct file name is accessed. The file can be downloaded from <https://github.com/theurbanpenguin/lfcs>

Demo



Create and Test Python Script

- /usr/bin/file_test.py
- test execution



AppArmor Profiles

AppArmor profiles are a set of rules that are applied to services and executables protecting your applications:

1. **Enforcing:** Rules are enforced and violations logged
2. **Complain:** No rule enforcement, violations logged
3. **Rules:** Rules make up the content of a profile and define what is allowed



```
$ aa-easyprof /usr/bin/file_test.py | sudo tee /etc/apparmor.d/usr.bin.file_test.py
$ sudo aa-status --enforced
$ sudo apparmor_parser -r /etc/apparmor.d/usr.bin.file_test.py
$ sudo aa-status --enforced
```

Creating AppArmor Profile

A profile template can be created with aa-easyprof. It writes to the screen so can be created as a standard user. This can be loaded into AppArmor using apparmor_parser.

```
$ file_test.py
/usr/bin/python3: can't open file '/usr/bin/file_test.py': [Errno 13] Permission denied
$ sudo journalctl -k -g 'apparmor="DENIED"'
$ sudo aa-notify -s 1 -v
```

Reading Denials

The profile has no rules, this means that nothing can be executed. We can see the denials with journalctl or from aa-notify

```
/usr/bin/file_test.py {  
    /usr/bin/file_test.py r,  
}
```

Simple But Incomplete Profile

We can manually edit the profile and reload it, but it is a tiresome task as many rules are likely to be needed

```
$ sudo aa-complain /usr/bin/file_test.py  
$ file_test.py  
$ sudo aa-logprof  
$ sudo aa-enforce /etc/apparmor.d/usr.bin.file_test.py
```

Setting Complain Mode and Log Profiling

Setting the profile to complain, the script can run in its entirety. Nothing is denied and DENIALS are logged. Reading the denials with aa-logprof we can correct errors.

Demo



Creating and Managing AppArmor Profiles



Summary



**In this module we have introduced
AppArmor**

- profiles
- logs
- modes



Congratulations!!!

